

SIMATS ENGINEERING ASSESSMENT TOOL 1

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems. (BL2, BL3)*

Assessment Tool Weightage: 40%

Dr. G. A. Senthil

QUESTIONS: 10

Total Marks: 50

Annexure A

Descriptive Experiment Exam

Duration: 2hrs

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim

To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program.

Requirements

- Python 3.10 or later
- Anaconda Navigator
- Gymnasium
- NumPy
- Matplotlib
- TensorFlow
- Keras

Installation Commands

`pip install gymnasium`

P.Sahith Sai
192425029

```
pip install numpy  
pip install matplotlib  
pip install tensorflow  
pip install keras
```

Algorithm

1. Install Python and Anaconda Navigator.
2. Install the required Reinforcement Learning libraries.
3. Import all required packages.
4. Verify successful installation.
5. Display the version numbers.
6. Execute a simple Python program.

Python Code

```
import gymnasium as gym  
import numpy as np  
import tensorflow as tf  
import keras  
import matplotlib.pyplot as plt  
print("Gymnasium Version :", gym.__version__)  
print("NumPy Version :", np.__version__)  
print("TensorFlow Version :", tf.__version__)  
print("Keras Version :", keras.__version__)  
print("\nReinforcement Learning Environment Installed Successfully!")
```

Sample Output

```
Gymnasium Version : 1.1.0  
NumPy Version : 2.0.1  
TensorFlow Version : 2.18.0  
Keras Version : 3.7.0
```

Reinforcement Learning Environment Installed Successfully!

Result

The Reinforcement Learning environment was successfully installed and verified.

Experiment 2: Familiarization with OpenAI Gym/ Gymnasium Environments

Aim

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and MountainCar using Gymnasium.

Algorithm

1. Import Gymnasium.
2. Create an environment.
3. Reset the environment.
4. Select random actions.
5. Execute actions.
6. Observe states and rewards.
7. Repeat until the episode ends.

Python Code

```
import gymnasium as gym
env = gym.make("FrozenLake-v1", render_mode="ansi")
state, info = env.reset()
print("Initial State:", state)
for step in range(10):
    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"Step {step+1}")
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)

    if terminated or truncated:
        print("Episode Finished")
        break
env.close()
```

Sample Output

Initial State: 0

Step 1
Action: 2
Next State: 1
Reward: 0

Step 2
Action: 1
Next State: 5
Reward: 0

Episode Finished

Result

Successfully interacted with the Gymnasium environment and understood states, actions, rewards, and episode termination.

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim

To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment.

Algorithm

1. Initialize the environment.
2. Reset the environment.
3. Observe the current state.
4. Choose a random action.
5. Perform the action.
6. Receive reward.
7. Move to the next state.
8. Repeat until completion.

Python Code

```
import gymnasium as gym
env = gym.make("FrozenLake-v1")
state, info = env.reset()
total_reward = 0
done = False
while not done:

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    total_reward += reward

    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward)
    print()

    state = next_state

    done = terminated or truncated
print("Total Reward:", total_reward)
env.close()
```

Sample Output

```
Current State: 0
Action: 1
Next State: 4
```

P.Sahith Sai
192425029
Reward: 0

Current State: 4
Action: 2
Next State: 8
Reward: 0

...

Total Reward: 1

Result

Successfully implemented the Reinforcement Learning interaction loop between the agent and the environment.

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim

To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution.

Algorithm

1. Define states.
2. Define actions.
3. Define transition probabilities.
4. Define rewards.
5. Initialize the current state.
6. Execute actions according to the policy.
7. Move to the next state.
8. Calculate total reward.

Python Code

```
import random
states = ["A", "B", "Goal"]
transitions = {
    "A": "B",
    "B": "Goal"
}
rewards = {
    "A": 0,
    "B": 5,
    "Goal": 10
}
current = "A"
total_reward = 0
while current != "Goal":
```

```
print("Current State:", current)

total_reward += rewards[current]

current = transitions[current]
print("Current State:", current)
total_reward += rewards[current]
print("Total Reward:", total_reward)
```

Sample Output

Current State: A
Current State: B
Current State: Goal

Total Reward: 15

Result

Successfully simulated a Markov Decision Process and observed state transitions and accumulated rewards.

Experiment 5: Bellman Equation Demonstration

Aim

To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value functions.

Algorithm

1. Initialize state values.
2. Define rewards.
3. Define discount factor.
4. Apply Bellman equation.
5. Update values repeatedly.
6. Stop after convergence.
7. Display final state values.

Python Code

```
gamma = 0.9
reward = [0, 5, 10]
V = [0, 0, 0]
iterations = 10
for i in range(iterations):

    V[2] = reward[2]

    V[1] = reward[1] + gamma * V[2]

    V[0] = reward[0] + gamma * V[1]
```

```

P.Sahith Sai
192425029
print("State Values")
for i in range(len(V)):
    print(f"State {i} :", round(V[i],2))

```

Sample Output

State Values

State 0 : 12.6
 State 1 : 14.0
 State 2 : 10

Result

Successfully demonstrated the Bellman Equation and computed state-value estimates using dynamic programming principles.

Code of Conduct:

I P.Sahith Sai (192425029) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P.Sahith Sai

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

P.Sahith Sai
192425029